

# Progetto di High Performance Computing

## Skyline Operator

Luca Venturi - Matricola 0000978350

Anno Accademico 2024/2025

## 1 Introduzione

L'operatore skyline individua tutti i punti non dominati da nessun altro punto in un insieme. Un punto  $p$  domina un punto  $q$  se  $p_i \geq q_i$  per tutte le dimensioni  $i$ , e  $p_j > q_j$  per almeno una dimensione  $j$ .

Questo progetto richiede di implementare due versioni parallele di un algoritmo seriale per calcolare l'operatore skyline; si sono usati `OpenMP` e `CUDA`.

### 1.1 Specifiche calcolatore

Tutti i test sono stati eseguiti su un calcolatore con:

- **CPU:** Intel i5-7600k con 4 core fisici, senza hyperthreading.
- **GPU:** Nvidia 1070

## 2 Algoritmo Seriale

L'algoritmo seriale per il calcolo dello skyline è molto semplice: si assume inizialmente che tutti i punti appartengano allo skyline e, scorrendo la lista dei punti, si eliminano quelli dominati da altri. In pratica, ogni punto viene confrontato con tutti gli altri e vengono esclusi dallo skyline i punti dominati.

Il costo computazionale è circa  $\mathcal{O}(N^2 \cdot D)$ , poiché si tratta di due cicli innestati (costo  $\mathcal{O}(N^2)$ ) ed ogni chiamata a *dominates* ha costo circa  $\mathcal{O}(D)$ .

Un dettaglio importante è che, quando un punto viene escluso, tutti i confronti che lo coinvolgono possono essere saltati. Ad esempio, se un punto  $P_1$  domina  $P_2$ , allora non è necessario confrontare  $P_2$  con altri punti, perché ogni punto dominato da  $P_2$  è dominato anche da  $P_1$ , quindi sono sicuro che verrà escluso quanto confrontato con  $P_1$ ; inoltre basta che sia dominato da un solo punto per poterlo escludere dallo skyline.

Questa caratteristica rende il tempo di esecuzione dell'algoritmo *output-dependent*: il numero effettivo di istruzioni eseguite dipende anche dal numero  $K$  di punti nello skyline.

### 2.1 Analisi dell'algoritmo

Prima di procedere all'implementazione delle versioni parallele è necessario analizzare il codice seriale della funzione `skyline`. È evidente che non è possibile parallelizzare nessuno dei due cicli direttamente, ci sono diverse criticità:

- Nel **ciclo interno**, più thread andrebbero a modificare concorrentemente la stessa variabile `r`, causando race condition.

- Nel **ciclo esterno**, oltre al problema di **r**, si hanno accessi concorrenti all'array **s**. Per esempio un thread potrebbe impostare  $s[j] = 0$  mentre un altro thread sta leggendo quello stesso valore.
- Inoltre, il ciclo esterno contiene una condizione: l'iterazione  $i$  viene eseguita solo se  $s[i]$  non è stato azzerato da iterazioni precedenti. Questa non è una vera e propria *loop-carried dependency*, poiché non influisce direttamente sul risultato finale, ma influenza fortemente il numero di confronti eseguiti.

## 3 Parallelizzazione con OpenMP

### 3.1 Discussione sull'algoritmo

La soluzione proposta sfrutta una sezione parallela creata con `#pragma omp parallel`, in modo da riutilizzare la stessa pool di thread sia per l'inizializzazione dell'array **s** sia per il calcolo dello skyline. L'inizializzazione di **s** è parallelizzata facilmente con una direttiva `#pragma omp for`. Nei cicli che calcolano lo skyline è invece necessario utilizzare una **critical section**. Nello specifico, si racchiudono la modifica di  $s[j]$  e il decremento di **r** con `#pragma omp critical`. Viene inoltre inserito un ulteriore controllo su  $s[j]$  per evitare decrementi multipli da parte di thread diversi, infatti non ho garanzie che  $s[j]$  non sia già stato impostato a 0 da un altro thread nel tempo che è passato tra il controllo precedente e la sezione critica. Una possibile alternativa sarebbe stata quella di spostare il calcolo di **r** in un ciclo separato, usando una riduzione, ma dai test questa variante si è dimostrata leggermente meno efficiente. Si è anche testata una versione che non faceva uso delle sezioni critiche, pensando fosse migliore, e usando la riduzione di array su **s** con l'operatore AND e la riduzione somma in un ciclo a parte per **r**. È risultata meno efficiente per due motivi:

1. Lavorando su **s** globale è possibile sfruttare la condizione `if (s[i])`, che consente di saltare molte iterazioni non necessarie.
2. La copia dell'array **s** dalla memoria globale a quella privata di ciascun thread risulta più onerosa rispetto al costo della sincronizzazione dovuta alla sezione critica.

Per quanto riguarda lo scheduling, il carico di lavoro è sbilanciato (proprio a causa di quella condizione) e si sono quindi testati vari chunksize sia per *dynamic* che per *guided*, sembra essere leggermente meglio con `schedule(dynamic,64)`, anche se le differenze con *guided* sono minime.

### 3.2 Discussione delle prestazioni

Si sono considerati solo i tempi di esecuzione della funzione skyline, dove la porzione di codice seriale è quasi lo 0% e quindi l'efficienza massima può raggiungere il 100% (legge di Amdal)

#### 3.2.1 Speedup

Lo speedup è stato calcolato come:

$$Speedup = \frac{T_{parallel}(1)}{T_{parallel}(p)} \quad \text{con } T_{parallel}(p) \text{ tempo di esecuzione con } p \text{ thread.}$$

Come si osserva in Figura 1, lo speedup cresce in modo quasi ideale fino a 4 thread sui dataset grandi, mentre per input più piccoli l'overhead dovuto alla sincronizzazione non è indifferente; con dataset molto piccoli l'implementazione parallela risulta addirittura più lenta della versione seriale.

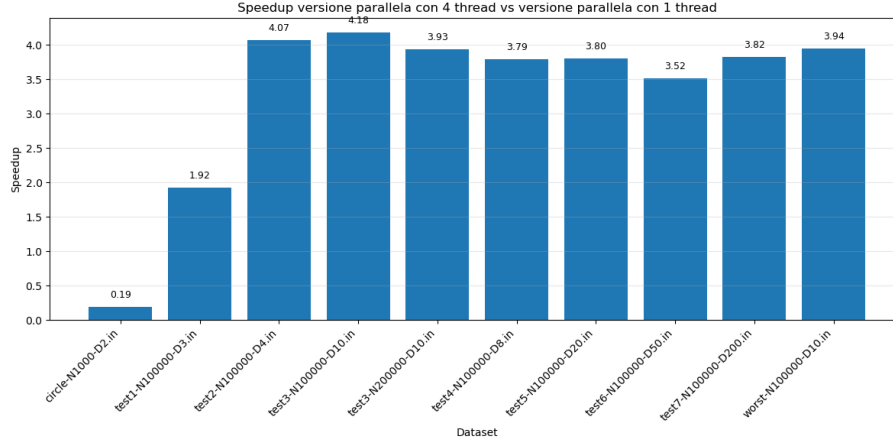


Figura 1: Speedup versione OMP eseguita con 4 thread calcolato rispetto all'esecuzione con 1 thread.

### 3.2.2 Efficiency

L'efficienza è definita come:

$$Efficiency = \frac{S_p}{p} \quad \text{con } S_p \text{ speedup e } p \text{ numero di thread.}$$

Il grafico in Figura 2 conferma quanto visto per lo speedup: efficienza elevata sui dataset grandi, ma in calo per input piccoli. Comunque ottimi risultati in quanto l'efficienza è vicina al 100%.

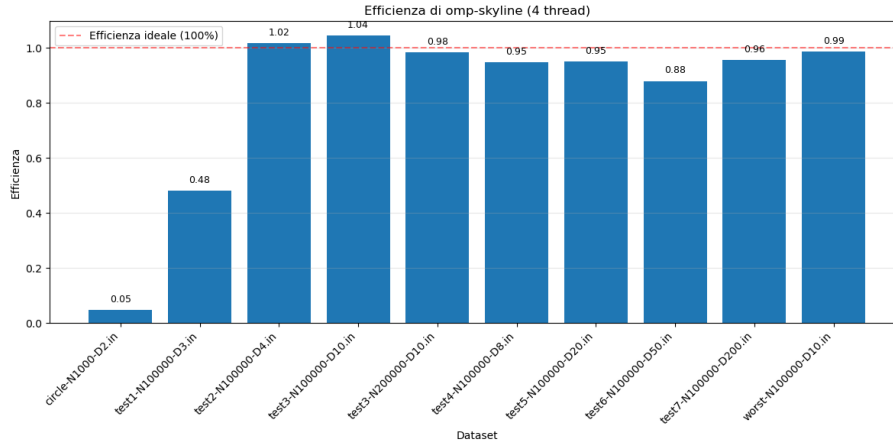


Figura 2: Efficienza dell'implementazione OpenMP calcolata con speedup del grafico precedente.

### 3.2.3 Strong Scaling Efficiency

La *Strong Scaling Efficiency* misura come varia il tempo di esecuzione al crescere del numero di thread, mantenendo fisso il dataset. I risultati sono riportati in Tabella 1 e Figura 3, e mostrano una scalabilità ideale.

### 3.2.4 Weak Scaling Efficiency

Infine, si è analizzata la *Weak Scaling Efficiency*, aumentando la dimensione del problema proporzionalmente al numero di thread in modo da mantenere costante il carico di lavoro per thread. Poiché l'algoritmo ha complessità  $\mathcal{O}(N^2)$ , visto che  $D \ll N$ , la dimensione del dataset è stata

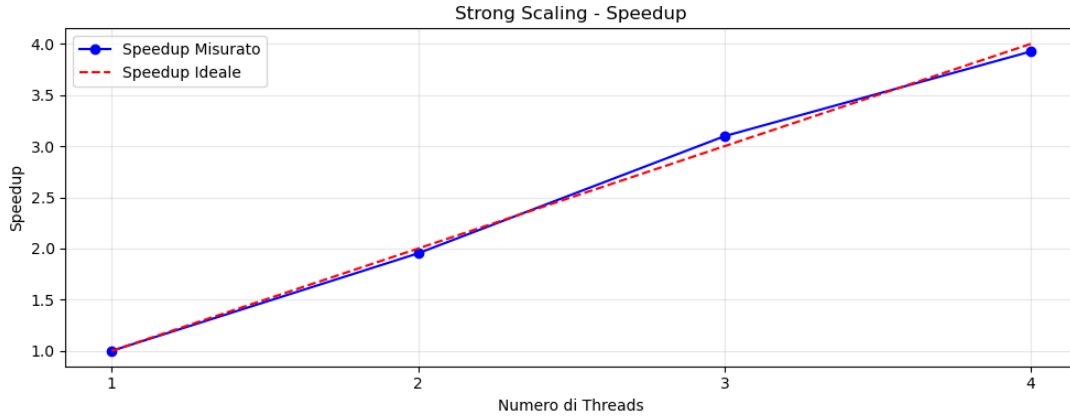


Figura 3: Speedup per l’implementazione OpenMP eseguita sullo stesso dataset e aumentando il numero di thread, confrontato con lo speedup ideale. Crescita lineare.

Tabella 1: Tempi di esecuzione, speedup ed efficienza in funzione del numero di thread. Dataset: N100000 punti, 10 dimensioni. 10 ripetizioni. 24892 punti nello skyline.

| Threads | N      | D  | K     | Tempo (s) | Speedup | Efficienza (%) |
|---------|--------|----|-------|-----------|---------|----------------|
| 1       | 100020 | 10 | 24892 | 18.87     | 1.00    | 1.00           |
| 2       | 100020 | 10 | 24892 | 9.66      | 1.95    | 0.98           |
| 3       | 100020 | 10 | 24892 | 6.09      | 3.10    | 1.03           |
| 4       | 100020 | 10 | 24892 | 4.81      | 3.93    | 0.98           |

scalata di un fattore  $\sqrt{p}$ , dove  $p$  è il numero di thread. I risultati riportati in Tabella 2 mostrano un’efficienza addirittura superiore a quella ideale. Questo effetto è spiegato dalla natura *output-dependent* dell’algoritmo ( $K$  aumenta di un fattore minore di  $\sqrt{p}$ ) e dal lavoro della cache, che rendono le esecuzioni parallele più efficienti del previsto.

Tabella 2: Tempo di esecuzione e weak scaling efficiency su dataset di dimensioni crescenti per rendere costante il carico di lavoro per thread.

| Threads | N      | D  | K     | Tempo (s) | Efficienza (%) |
|---------|--------|----|-------|-----------|----------------|
| 1       | 100000 | 10 | 24892 | 19.27     | 100            |
| 2       | 141421 | 10 | 31212 | 17.66     | 109            |
| 3       | 173205 | 10 | 34688 | 15.23     | 127            |
| 4       | 200000 | 10 | 39965 | 17.40     | 111            |

## 4 Parallelizzazione con CUDA

Come seconda tecnologia di parallelizzazione è stata scelta CUDA, principalmente per la disponibilità di una scheda NVIDIA e la volontà di sfruttarla, ma anche perché il problema si presta bene ad essere parallelizzato su GPU: la funzione `dominates` è infatti un’operazione banale (costo  $\mathcal{O}(D)$ , con  $D \ll N$ ), che può essere facilmente replicata in parallelo su migliaia di thread.

### 4.1 Discussione dell’algoritmo

Si sono sviluppate due varianti dell’algoritmo, che differiscono nel modo in cui mappano il lavoro sui thread:

**Versione 1D** ad ogni thread viene assegnato un punto  $P_i$ . Il thread cicla su tutti gli altri punti, se  $P_i$  domina qualche altro punto  $P_j$ , aggiorna  $s[j]$  tramite `atomicExch`, impostandolo a 0. Questa versione richiede  $N$  thread ed è più fedele alla logica del ciclo seriale.

**Versione 2D** ad ogni thread viene assegnata direttamente una coppia di punti  $(P_i, P_j)$ . In pratica, il doppio ciclo annidato viene "srotolato" e valutato in parallelo. Ogni thread esegue il test di dominanza su una singola coppia e in caso aggiorna  $s[j]$  con `atomicCAS`. Questa strategia sfrutta maggiormente la parallelizzazione massiccia della GPU, ma richiede fino a  $N^2$  thread. Per valori grandi di  $N$  la GPU suddivide l'esecuzione in più batch e c'è molto overhead.

Inizialmente è stata implementata la versione 1D, più intuitiva. Successivamente, ricercando metodi efficienti per parallelizzare due loop annidati, ho trovato sul forum NVIDIA la soluzione alternativa della griglia bidimensionale, a cui non avevo inizialmente pensato e che ho voluto provare ad implementare. Questa variante "extra" si è rivelata particolarmente utile con input ad alta dimensionalità. Anche qui, come per OMP, si usano operazioni atomiche per impostare a 0  $s[j]$  in modo da evitare contese tra thread. L'inizializzazione di  $s$  e il calcolo di  $r$  sono fatti lato host: si è testata una versione che usa altri due kernel per queste computazioni, ma il guadagno di efficienza per dataset grandi non vale l'overhead sui dataset più piccoli; considerando 100.000 punti come caso d'uso medio è una complicazione inutile e quasi peggiorativa.

## 4.2 Discussione delle prestazioni

Anche per CUDA sono considerati solo i tempi di esecuzione della funzione skyline.

### 4.2.1 Speedup CUDA rispetto alla versione seriale

Come si può verificare nella Tabella 3, sui dataset piccoli, l'overhead dovuto ad inizializzazione e trasferimento dati verso la GPU pesa più del calcolo effettivo, portando a speedup molto bassi, mentre per dataset di dimensioni maggiori entrambe le versioni producono ottime velocizzazioni. Si notano anche le principali differenze tra le due versioni: la versione 1 soffre con  $D$  troppo grandi, mentre la versione 2 scala meglio con la dimensionalità del problema. Nel caso di test peggiore dove  $K \approx N$  lo speedup della versione 1 è quasi il doppio della versione 2.

Tabella 3: Confronto dei tempi di esecuzione e speedup delle versioni CUDA rispetto alla versione seriale su diversi dataset.

| Dataset               | Tempo (s) |       | Speedup       |               |
|-----------------------|-----------|-------|---------------|---------------|
|                       | CUDA1     | CUDA2 | CUDA1         | CUDA2         |
| circle-N1000-D2.in    | 0.05      | 0.02  | 0.00          | <b>0.01</b>   |
| test1-N100000-D3.in   | 0.41      | 0.18  | 0.06          | <b>0.14</b>   |
| test2-N100000-D4.in   | 0.35      | 0.34  | 7.36          | <b>7.52</b>   |
| test3-N100000-D10.in  | 0.41      | 0.67  | <b>49.89</b>  | 31.02         |
| test4-N100000-D8.in   | 0.34      | 0.52  | <b>38.33</b>  | 25.28         |
| test5-N100000-D20.in  | 1.12      | 1.18  | <b>81.23</b>  | 77.45         |
| test6-N100000-D50.in  | 2.38      | 1.34  | 55.17         | <b>98.26</b>  |
| test7-N100000-D200.in | 4.00      | 1.34  | 48.90         | <b>145.52</b> |
| worst-N100000-D10.in  | 0.51      | 0.87  | <b>115.30</b> | 67.62         |

### 4.2.2 Analisi dei risultati

Quando  $D$  aumenta troppo allora la funzione dominates non è più un'operazione banale, diventa un calcolo oneroso per un cuda core che lavora a frequenze di clock più basse rispetto a una

normale CPU. In questo modo la versione 1, in cui ogni thread esegue dominates circa  $N$  volte, perde di efficienza. La versione 2 sfrutta meglio la distribuzione massiccia disponibile con CUDA, ma ha un alto overhead dovuto all'alto numero di thread richiesti e alle operazioni atomiche, quando  $D$  è piccolo e la funzione dominates è veloce il costo principale diventa la gestione dei thread e delle atomiche.

### 4.2.3 Dataset con $N$ più grandi

Sono stati generati dataset da 1.000.000 punti per verificare la scalabilità delle due versioni all'aumentare di  $N$ , come mostrato in Tabella 4. I risultati rafforzano quanto osservato prima e ci dicono che la versione 1 scala meglio all'aumentare di  $N$ , mentre quando  $D$  aumenta troppo la versione 2 prende il sopravvento.

Tabella 4: Tempi di esecuzione delle versioni CUDA su dataset da 1.000.000 punti.

| Dataset               | CUDA1 (s)    | CUDA2 (s)     |
|-----------------------|--------------|---------------|
| test1-N1000000-D3.in  | <b>4.05</b>  | 15.97         |
| test2-N1000000-D4.in  | <b>11.70</b> | 35.77         |
| test3-N1000000-D10.in | <b>16.12</b> | 48.58         |
| test4-N1000000-D8.in  | <b>7.50</b>  | 22.75         |
| test5-N1000000-D20.in | <b>95.03</b> | 105.37        |
| test6-N1000000-D50.in | 226.19       | <b>124.33</b> |

### 4.3 Confronto CUDA vs OMP

Tabella 5: Confronto dello speedup delle versioni CUDA rispetto a OpenMP eseguito con 4 thread.

| Dataset               | Speedup rispetto a OMP |              |
|-----------------------|------------------------|--------------|
|                       | CUDA1                  | CUDA2        |
| circle-N1000-D2.in    | 0.01                   | <b>0.02</b>  |
| test1-N100000-D3.in   | 0.03                   | <b>0.06</b>  |
| test2-N100000-D4.in   | 1.71                   | <b>1.75</b>  |
| test3-N100000-D10.in  | <b>11.78</b>           | 7.32         |
| test4-N100000-D8.in   | <b>9.01</b>            | 5.94         |
| test5-N100000-D20.in  | <b>19.52</b>           | 18.61        |
| test6-N100000-D50.in  | 15.22                  | <b>27.10</b> |
| test7-N100000-D200.in | 12.51                  | <b>37.24</b> |
| worst-N100000-D10.in  | <b>27.50</b>           | 16.13        |

## 5 Conclusioni

Entrambe le implementazioni, OpenMP e CUDA, calcolano correttamente lo skyline, ma con caratteristiche di prestazioni differenti. L'implementazione OpenMP risulta semplice da sviluppare, con buoni tempi di esecuzione e ottima scalabilità, limitata solo dal numero di core fisici disponibili. La versione CUDA sfrutta invece il parallelismo massivo della GPU: la variante 1D è più efficiente per dataset con numero elevato di punti ma dimensioni moderate, mentre la variante 2D mostra vantaggi quando il numero di dimensioni cresce. A meno di dataset con  $D$  molto grande ( $> 50$ ) la versione 1 è più solida.